Протокол виконання лабораторної роботи №4

з криптографії

Виконали студенти 3 курсу, НН ФТІ, ФБ-43

Закусило Дмитрій, Топчій Микита

Варіант 6

КРИПТОГРАФІЯ
КОМП’ЮТЕРНИЙ ПРАКТИКУМ №4

Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем.

Мета та основні завдання роботи

Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

Порядок і рекомендації щодо виконання роботи

Завдання 1. Написати функцію пошуку випадкового простого числа з заданого інтервалу або заданої довжини, використовуючи датчик випадкових чисел та тести перевірки на простоту. В якості датчика випадкових чисел використовуйте вбудований генератор псевдовипадкових чисел вашої мови програмування. В якості тесту перевірки на простоту рекомендовано використовувати тест Міллера-Рабіна із попередніми пробними діленнями. Тести необхідно реалізовувати власноруч, використання готових реалізацій тестів не дозволяється.

Доповнення до п. 1. Для одного зі згенерованих простих чисел навести проміжні результати перевірки тестом Міллера-Рабіна для однієї з використаних основ.

Пояснення коду програми Task1.py

mod_pow призначена для швидкого модульного піднесення числа до степеня за логарифмічний час. Показник розглядається як сума степенів двійки, основа на кожному кроці підноситься до квадрата, а множення на накопичений результат

виконується лише для тих розрядів, де двійковий біт дорівнює одиниці. Усі операції на кожному етапі беруться за заданим модулем.

Функція ініціалізує змінну result = 1 та зменшує основу base за модулем. Далі іде цикл, який ітерується за двійковими розрядами числа: побітова операція if exponent & 1 перевіряє молодший біт, і якщо він дорівнює одиниці, змінна result множиться на поточне значення base за модулем. Після цього основа підноситься до квадрата для переходу до наступного розряду степеня, а сам показник зміщується праворуч на один розряд, що рівне цілочисельному діленню на 2. Цикл завершується, коли показник стає рівним 0.

Функція miller_rabin реалізує імовірнісний тест перевірки непарного числа «p» на простоту. Алгоритм розкладає p-1 на непарну частку dта степінь двійки 2^s, формуючи ланцюжок піднесень до квадрата для випадково обраної основи x: якщо найперший член x^d mod p= +-1, або якщо один із наступних квадратів у ланцюжку дає -1 перед появою одиниці, число вважається сильно псевдопростим за цією основою. Якщо ж у ланцюжку з'являється 1 без попереднього значення -1, число складене. Повторення цієї процедури для k=40 незалежних основ зменшує ймовірність помилки до величини не більше 4^(-40).

Синтаксична реалізація miller_rabin спершу обробляє граничні випадки, після чого через цикл ділить число d на 2 і збільшує лічильник s, поки d не стане непарним. Основний цикл виконує k раундів: у кожному генерується випадкова основа x є[2,p-2], перевіряється gcd(x,p)=1, та обчислюється стартове значення xr. Якщо xr == 1 або xr == p - 1, поточний раунд завершується успішно, інакше у вкладеному циклі for виконується послідовне піднесення xr^2 . Якщо отримано xr == p – 1 то раунд пройдено, а якщо виникає передчасна xr == 1 або вкладений цикл вичерпується без виходу на p-1, функція повертає False.

Інші функції файлу є допоміжними: trial_division відсіює кандидата звичайним діленням за масивом простих чисел до 541, generate_prime_candidate повертає випадкове непарне число потрібної довжини із встановленими старшим і молодшим бітами, is_prime об'єднує пробне ділення та тест Міллера-Рабіна в

послідовний ланцюг, а generate_prime організовує цикл генерації та відбору кандидатів до повернення першого числа, яке пройшло всі перевірки.

Демонстрація роботи програми

Перший запуск

```
PS C:\Users\user\Desktop\Cryptography\Lab4> python -u "c:\Users\user\Desktop\Cryptography\Lab4\Task1.py"
Generated 256-bit prime:
115469428794181794898391252359667153517180300233520795059133675658298212823281

Intermediate Miller-Rabin test steps for this prime:
p - 1 = 2^4 * 7216839299636362181149453272479197094823768764595049691195854728643638301455
Base x: 139065575774597077675514779814864077918780128626316493798154260289369058463 60
x^d mod p = 75530664861454097847339775767000310087157390951141255111549023234586693693010
x^(d * 2^1) mod p = 948847274823961920891486696168964963466859065613859253873563261878524289674 83
x^(d * 2^2) mod p = 115469428794181794898391252359667153517180300233520795059133675658298212823280

Rejected candidates sample:
997717460424092194255354309458349553756011611129388415757862203479980269584 59 - Failed trial division
604017856264093215966197925935059038860928505985637360727325887725274811695 27 - Failed trial division
908691304456473366962054876089158042322410593268680273553821753927725934324 53 - Failed trial division
611093559030123269746535060599767632968150043657009314484432536189074287386 09 - Failed trial division
100264941608855833085535239997475937272921133210891075922571517216276051936533 - Failed Miller-Rabin
```

Другий Запуск

```
PS C:\Users\user\Desktop\Cryptography\Lab4> python -u "c:\Users\user\Desktop\Cryptography\Lab4\Task1.py"
Generated 256-bit prime:
899749115538419549023967393093101592687333912544522017213960856681976026838 61

Intermediate Miller-Rabin test steps for this prime:
p - 1 = 2^2 * 2249372788846048872559918482732753981718334781361305043034902141704940067 0965
Base x: 1116081604230122130483276696868570420705187433548648878470633918011433827804 6
x^d mod p = 2534336138550321769630254786291934149099513589952376723522850075009104607896 1
x^(d * 2^1) mod p = 899749115538419549023967393093101592687333912544522017213960856681976026838 60

Rejected candidates sample:
973427242483818449336431698186477627300777606235384418600636409941018264311 61 - Failed trial division
109392163822749495724261759645501223501652561386374956973803991427694010215743 - Failed Miller-Rabin
919796563838148841837262364632422038199667574760192025585471909260783555763 07 - Failed trial division
800146980778827964526577530531603711978802043473658968953693251789657857934 05 - Failed trial division
748931421255774037517722256230237503400359442272887402335356453585915250783 37 - Failed Miller-Rabin
```

Аналіз отриманих результатів

Видно, що тест пробних ділень відсіяв переважну більшість складених кандидатів без запуску важких модульних операцій. Тест Міллера-Рабіна відкинув складені числа, дільники яких перевищують малі прості числа.

Для кандидатів s>1 тест Міллера-Рабіна успішно ітерує ланцюжок піднесень до квадрата, фіксує виконання сильної псевдопростоти в момент появи значення p-1. Крім того, наявні й кандидати які не пройшли тест, наведені в кінці, тобто тест відсіює складені числа.

Завдання 2. За допомогою цієї функції згенерувати дві пари простих чисел p, q і p_1, q_1 довжини щонайменше 256 біт. При цьому пари чисел беруться так, щоб $pq \leq p_1q_1$; p і q – прості числа для побудови ключів абонента A, p_1 і q_1 - абонента B.

Пояснення коду програми Task2.py

Програма використовує генерацію простих чисел з 1-го завдання для створення двох незалежних простих чисел p і q для абонента A та p1 і q1 для абонента B, контролюючи їхню попарну нерівність. Далі обчислюються добутки n, n1 та перевіряється співвідношення n <= n1: якщо модуль абонента A перевищує модуль абонента B, сформовані пари міняються місцями.

Демонстрація роботи програми

```
PS C:\Users\user\Desktop\Cryptography\Lab4> python -u
"c:\Users\user\Desktop\Cryptography\Lab4\Task2.py"
Subscriber A:
p=6379788324531884819716102665315558396140147420109615386341669466512856397790 1
q=6459316945098809976946843172654019423554454663090671533775340694660527759457 9
n=4120907483079234954605322680376420327174686941112762708639579798758354740302581 9
8212108735259117922872486804532708541084778490450485292301667442499339867 9
Bits(n)=511
Subscriber B:
p1=6564884350555215912119169403752254119340145580072927989223598474182604067497 3
q1=9126653949830921835376275819213919087263081095947053953071462826410219763420 3
n1=5991542768817796740048638490892454549588186500455066247707340983245747048962920
9377385885629178126492449029197659240342115096462486379703360326898709015 19
Bits(n1)=511
Verification:
Condition n <= n1: True
Difference (n1 - n) >= 0: True
```

Аналіз отриманих результатів

Обидва згенеровані модулі n та n1 мають довжину 511–512 біт, що відповідає добутку двох незалежних 256-бітних чисел. Перевірка умови n <= n1 повертає значення True, n1-n>=0, параметри підібрані правильно: будь-яке число або цифровий підпис за модулем n завжди менше за модуль n1 і не буде спотворене при наступному шифруванні на відкритому ключі абонента B.

Завдання 3. Написати функцію генерації ключових пар для RSA. Після генерування функція повинна повертати та/або зберігати секретний ключ (d, p, q) та відкритий ключ (n, e). За допомогою цієї функції побудувати схеми RSA для абонентів A і B - тобто, створити та зберегти для подальшого використання відкриті ключі (e, n), (e_1, n_1) та секретні d і d_1.

Для однієї зі згенерованих пар ключів навести значення основних параметрів та перевірити виконання співвідношень, необхідних для коректної роботи RSA.

Пояснення коду програми Task3.py

GenerateKeyPair реалізує процедуру створення ключових пар RSA: обчислює спільний модуль n = p * q та значення функції Ейлера phi = (p - 1) * (q - 1). Далі алгоритм перевіряє умову взаємної простоти gcd(e, phi) == 1, цим гарантуючи коректність побудови криптосистеми, і за допомогою функції mod_inverse обчислює секретну частину d. У результаті функція повертає відкритий ключ (e, n) та секретні дані (d, p, q).

У функції main за допомогою generate_both_pairs створює набори простих чисел для двох абонентів із дотриманням умови n <= n1, після чого викликає GenerateKeyPair окремо для абонента A та абонента B. Виводить у термінал відкриті ключі (e, n), (e1, n1), закриті ключі d, d1 та значення функцій Ейлера. Наприкінці блок виконує три перевірки для абонента A: перевіряє взаємну простоту e та phi(n), підтверджує мультиплікативну оберненість рівністю (e * d) mod phi(n) == 1, а також здійснює тестове шифрування й розшифрування числа через mod_pow.

Демонстрація роботи програми

```
PS C:\Users\user\Desktop\Cryptography\Lab4> python -u
"c:\Users\user\Desktop\Cryptography\Lab4\Task3.py"
Subscriber A Keys:
Public Key (e, n):
e = 65537
n =
50087913791372054701268625623591722638868785011548436949014334299371734985535418
56377145874927866373207322255023427470132039790801723892713970952174778687
Private Key d =
29757587490743584873408840918567638321360376782728076369788396177431647365560705
26721338274297228705063996333669986620415553589359050298188405891344534081
```

phi(n) =
50087913791372054701268625623591722638868785011548436949014334299371734985534003007431771749182627297046211146427448413338333569402116137346814490714693360

Subscriber B Keys:
Public Key (e1, n1):
e1 = 65537
n1 =
86096598934686829682661848828380764011871210690686230070436481134573700127384426995121246066756754232565957097413380883074860356468830844233736212177762670
Private Key d1 =
31689307711712707411159636807272239948339981144708076990998501578164804529849124606728406803820186997594226110990538172384597365601122525534555270224014090
phi(n1) =
86096598934686829682661848828380764011871210690686230070436481134573700127382558633245982783432700242987015862531585283291988954041985198406357215184114560

Verification for Subscriber A:
1. gcd(e, phi(n)) == 1: True
2. (e * d) mod phi(n) == 1: True
3. Decryption test (m^e)^d mod n == m: True

Аналіз отриманих результатів

Згенеровані ключові пари відповідають вимогам RSA: модулі n та n1 мають довжину, при цьому значення модуля абонента А є меншим за модуль абонента В, n <= n1. При шифруванні використовували e = e1 = 65537, секретні d та d1 є повністю різними числами, бо залежать від p,q.

Усі три контрольні перевірки повернули значення True: експонента e є взаємно простою з phi(n), обчислений d є правильним оберненим елементом, а тестове повідомлення після послідовного піднесення до степеня e, а потім до степеня d за модулем n відновилося без помилок. Отже параметри для шифрування-розшифрування формуються правильно.

Завдання 4. Написати програму шифрування, розшифрування і створення повідомлення з цифровим підписом для абонентів A і B. Кожна з операцій (шифрування, розшифрування, створення цифрового підпису, перевірка цифрового підпису) повинна бути реалізована окремою процедурою, на вхід до якої повинні подаватись лише ті ключові дані, які необхідні для її виконання.

За допомогою датчика випадкових чисел вибрати відкрите повідомлення M і знайти криптограму для абонентів A і B, перевірити правильність розшифрування. Скласти для A і B повідомлення з цифровим підписом і перевірити його.

Пояснення коду програми Task4.py

Encrypt обчислює криптограму M^e mod n на відкритому ключі адресата, тоді як Decrypt повертає відкритий текст через обчислення C^d mod n за допомогою його секретного ключа. Sign накладає цифровий підпис обчисленням M^d mod n на особистому закритому ключі автора, а Verify відновлює підписане значення відкритим ключем (e, n) автора через S^e mod n і звіряє його із вхідним повідомленням.

У __main__ для тестування генерується випадкове 512-бітне число M. Скрипт послідовно запускає повні цикли шифрування-розшифрування та підписання-перевірки окремо для абонента A та абонента B, виводячи отримані числові криптограми та підписи. Наприкінці виконується штучна модифікація повідомлення шляхом додавання одиниці до відкритого тексту, функція Verify повертає False, бо підпис не відповідає зміненому тексту.

Демонстрація роботи програми

PS C:\Users\user\Desktop\Cryptography\Lab4> python -u "c:\Users\user\Desktop\Cryptography\Lab4\Task4.py"

Original message M:
25585752010026492638736243055973876985951987294664162703981921700205045607515446704011342401391881967305460168042978199061610775199713261243304480501946 00

Subscriber A Operations

Ciphertext C_A:
13129263293038654688066525336987652582235960033074457151387383165597617665467927676947149478911474458303832630477262463514361543047506681857353206409576 22

Decryption success for A: True

Signature S_A:
35623938326624064793491317614728254581626394859558588385472878733549348569329164
06382278289824904121547202444675173294852951136345591708543427709845315136
Signature verification for A: True

Subscriber B Operations
Ciphertext C_B:
28604597511822571422595182442190157032093293796277978871854248902011440374207622
14931720709617851443679377379722738134886572339586853493936313143582995202
Decryption success for B: True
Signature S_B:
32372664192843673954896653883131425737763467805794603814802582898065710420458102
96260312650565624803799370740501967906201041018300789279921625496377363966
Signature verification for B: True

Tampering Check
Verification of tampered message: False

Аналіз отриманих результатів

Для обох абонентів зашифровані тексти C_A та C_B суттєво відрізняються від вихідного повідомлення M, проте операція розшифрування на відповідних закритих ключах d та d1 відновила початкове число. Значення цифрових підписів S_A та S_B також є унікальними 512-бітними числами, сформованими за допомогою закритих ключів кожного автора.

Успішна верифікація штатних підписів означає, що відкритий ключ автора підтверджує авторство без розкриття закритого ключа. Тест модифікації, що повернув False, підтверджує цілісность: зміна повідомлення навіть на найменше значення робить старий підпис недійсним, унеможливлюючи спотворення або підробку даних сторонньою особою.

Завдання 5. За допомогою раніше написаних на попередніх етапах програм організувати роботу протоколу конфіденційного розсилання ключів з підтвердженням справжності по відкритому каналу за допомогою алгоритму RSA. Протоколи роботи кожного учасника (відправника та приймаючого) повинні бути реалізовані у вигляді окремих процедур, на вхід до яких повинні подаватись лише ті ключові дані, які необхідні для виконання. Перевірити роботу програм для випадково обраного ключа $0 < k < n$.

У звіті для одного виконання протоколу показати послідовність переданих та обчислених значень і зазначити, використанням якого ключа забезпечується конфіденційність та підтвердження справжності відправника.

Кожна з наведених операцій повинна бути реалізована у вигляді окремої процедури, інтерфейс якої повинен приймати лише ті дані, які необхідні для її роботи; наприклад, функція Encrypt(), яка шифрує повідомлення для абонента, повинна приймати на вхід повідомлення та відкритий ключ адресата (і тільки його), повертаючи в якості результату шифротекст. Відповідно, програмний код повинен містити сім високорівневих процедур: GenerateKeyPair(), Encrypt(), Decrypt(), Sign(), Verify(), SendKey(), ReceiveKey().

Кожну операцію рекомендується перевіряти шляхом взаємодії із тестовим середовищем, розташованим за адресою:

http://asymcryptwebservice.appspot.com/?section=rsa.

Наприклад, для перевірки коректності операції шифрування необхідно: а) зашифрувати власною реалізацією повідомлення для серверу та розшифрувати його на сервері; б) зашифрувати на сервері повідомлення для вашої реалізації та розшифрувати його локально. До звіту додати скріншоти результатів перевірки взаємодії з тестовим середовищем.

Пояснення коду програми Task5.py

Програма організовує роботу розподілу ключів через дві процедури: SendKey для відправника та ReceiveKey для одержувача.

SendKey контролює виконання умов $0 < k < n$ та $n <= n1$, після чого накладає підпис на секретний сеансовий ключ особистим закритим ключем відправника

та окремо зашифровує сам ключ і сформований підпис відкритим ключем адресата. Сформована пара (k1, S1) містить необхідні дані для захищеної передачі відкритим каналом.

ReceiveKey на боці абонента B використовує його закритий ключ d1 для зняття шифрування з обох компонентів пакета за модулем n1, відновлюючи вихідні значення k і S, після чого застосовує процедуру Verify для перевірки автентичності підпису за допомогою відкритого ключа абонента A (e, n).

Блок __main__ демонструє як штатну передачу випадкового 512-бітного ключа, так і спробу компрометації каналу зв'язку: модифікація зашифрованого ключа призводить до отримання хибного числа після розшифрування, що викликає збій верифікації підпису та повернення False.

Демонстрація роботи програми

```
PS C:\Users\user\Desktop\Cryptography\Lab4> python -u
"c:\Users\user\Desktop\Cryptography\Lab4\Task5.py"
Generated secret session key k:
k =
5204138617316569848841308222437859855583569016880910749726004351000921288448910071679245805022954634158499150420415463480391836101854462825095699972945338

Sender A: SendKey
Encrypted key k1:
8067334897865378316143147859605785449410566778760862811426731920487072235239205846079921246073262466002007961947133288619260535501517596702447259359763771
Encrypted signature S1:
2818058318082051536017884342446558502176211196574112934371859281767025783677357189324050657640341715540327323572450858347873839799002837600658280701565383

Receiver B: ReceiveKey
Decrypted key k:
5204138617316569848841308222437859855583569016880910749726004351000921288448910071679245805022954634158499150420415463480391836101854462825095699972945338
Signature valid: True
Key match (received_k == k): True

Channel Tampering Simulation
Tampered transmission signature valid: False
```

Аналіз отриманих результатів

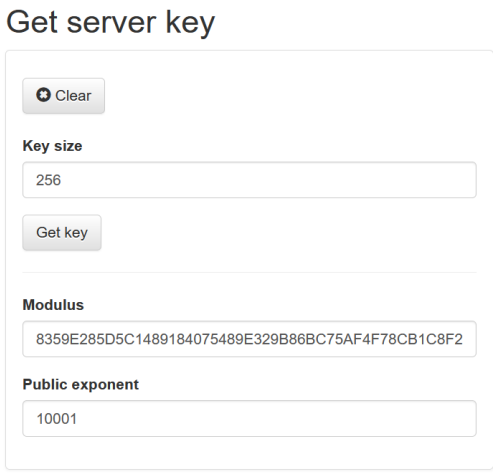
Результати підтверджують забезпечення двох основних властивостей безпеки: конфіденційності та автентичності. Конфіденційність гарантується тим, що значення k та S передаються зашифровані відкритим ключем адресата (e1, n1), і можуть бути прочитані лише власником закритого ключа d1.

Автентичність відправника та цілісність повідомлення гарантуються цифровим підписом S, створеним закритим ключем автора d.

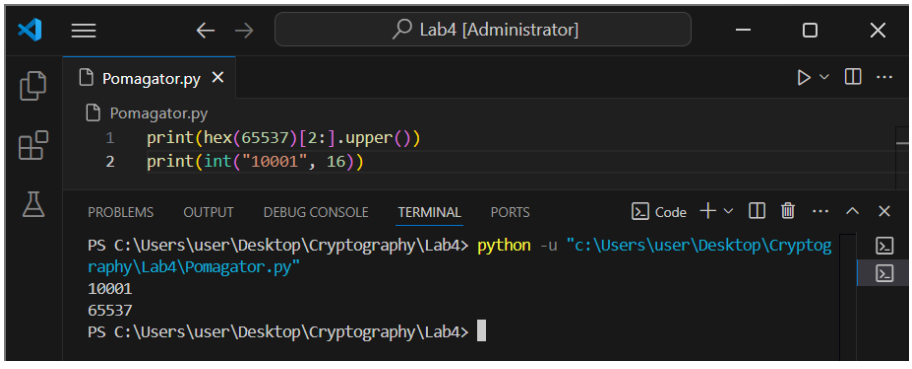
Блок перевірки втручання показав: спотворення хоча б одного розряду криптограми k1 призвело до порушення рівності S^e mod n == k після розшифрування, що не дало можливості непомітного нав'язування стороннього чи пошкодженого ключа.

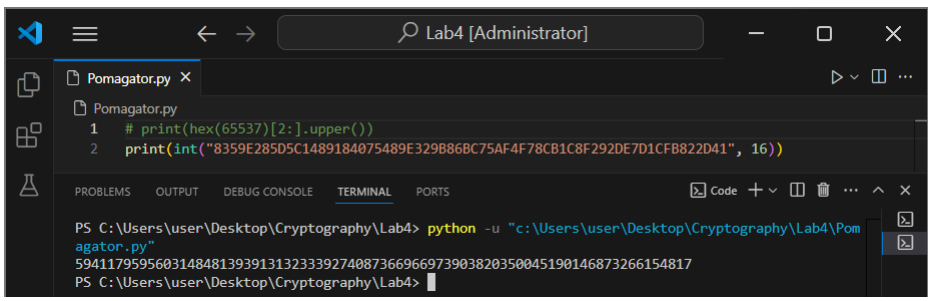
Виконаємо перевірку в тестовому середовищі

Зайдемо на середовище, введемо значення розміру ключа, отримаємо значення n, e.

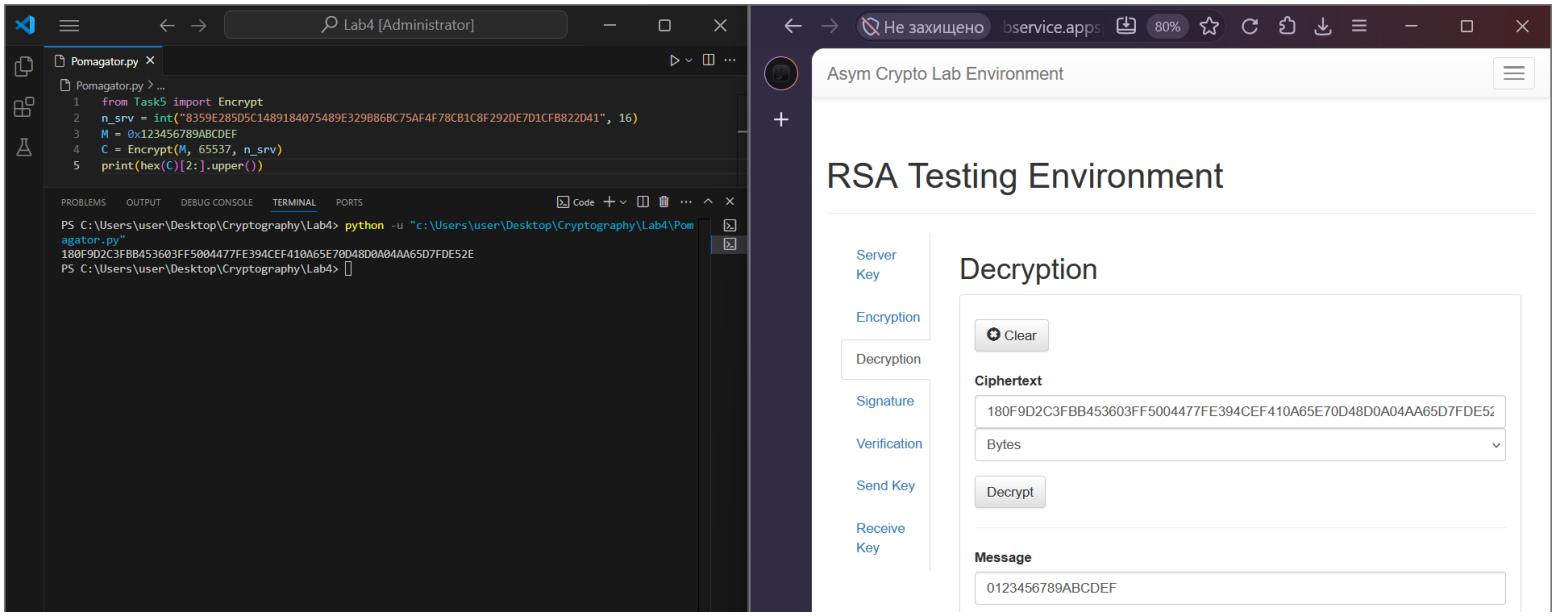


Ряд значень надаються у форматі hex, тому тут і надалі будемо для переведення між десятковою та шістнадцятковою системами будемо користуватись відповідними викликами в пайтоні:

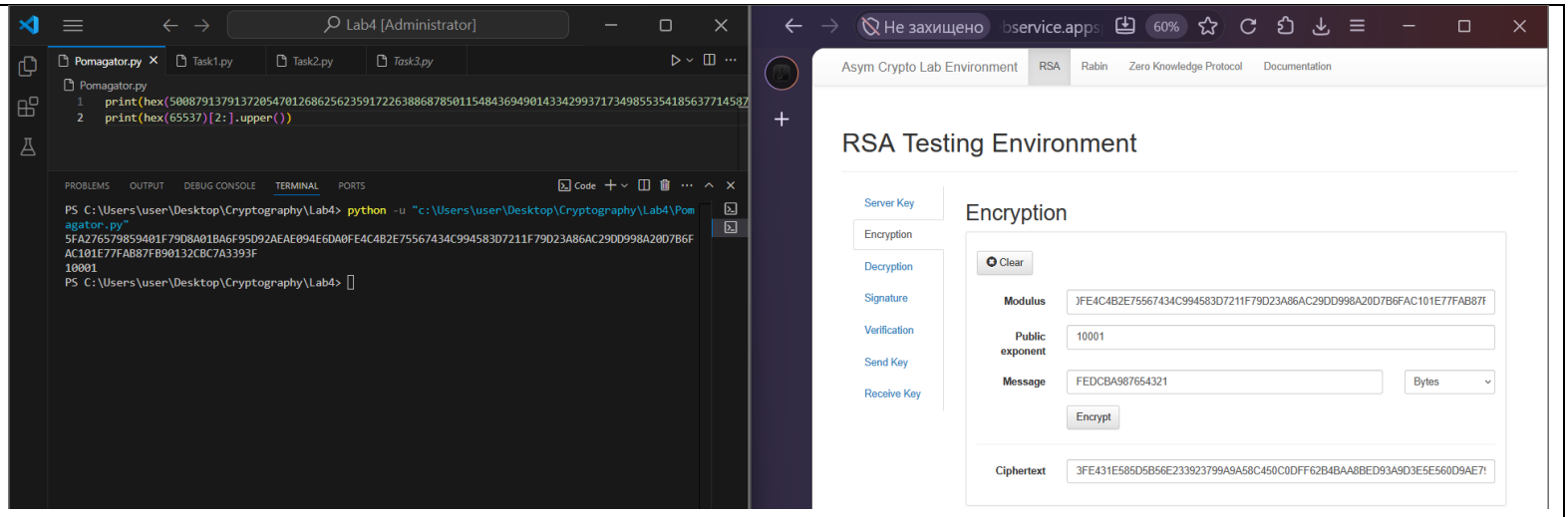




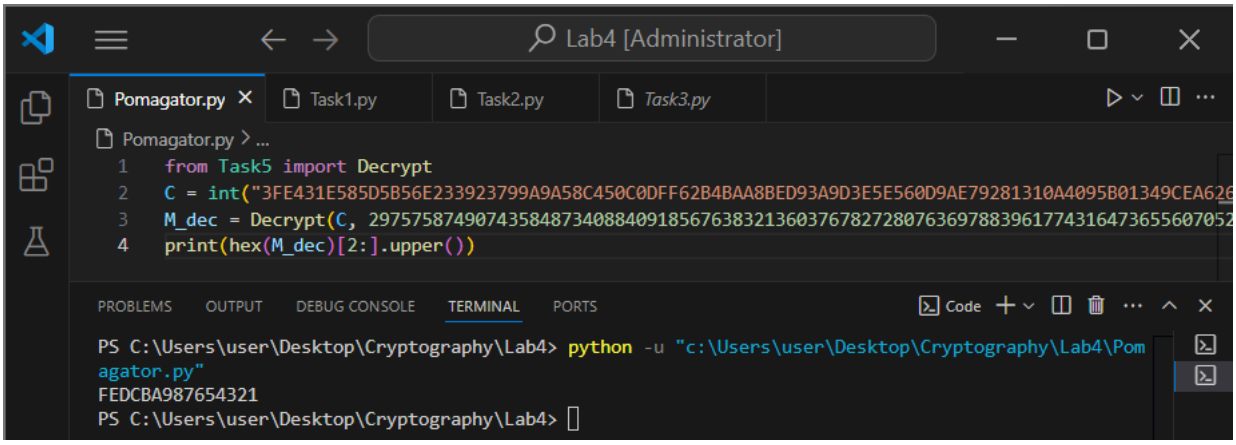
Зашифруємо повідомлення реалізованою нами функцією в Завданні5, розшифруємо повідомлення на сервері:



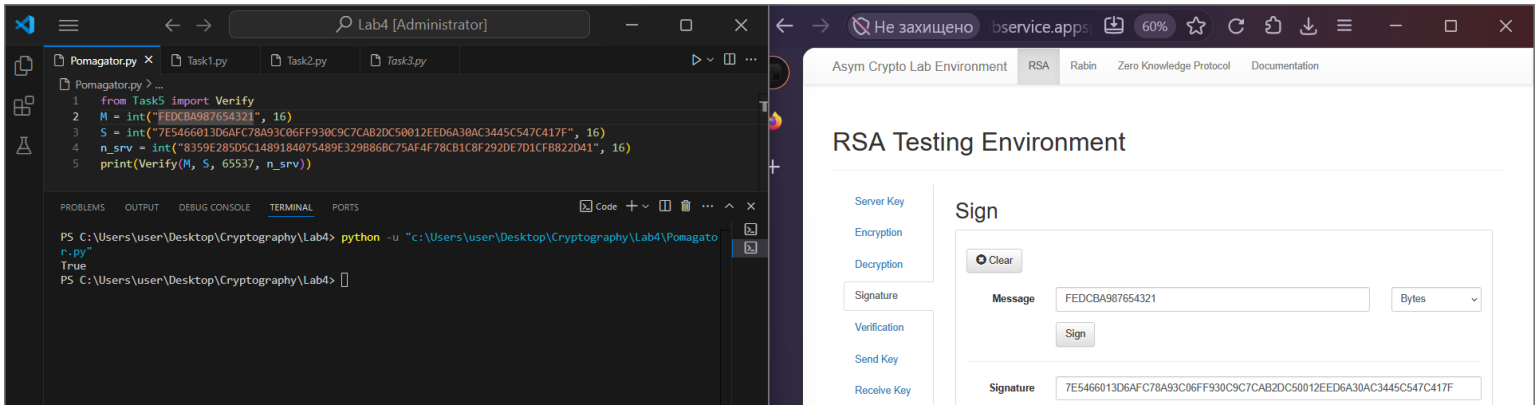
Зашифруємо повідомлення на сервері, давши йому згенеровану в завданні3 публічну пару (n,e) для абонента А:



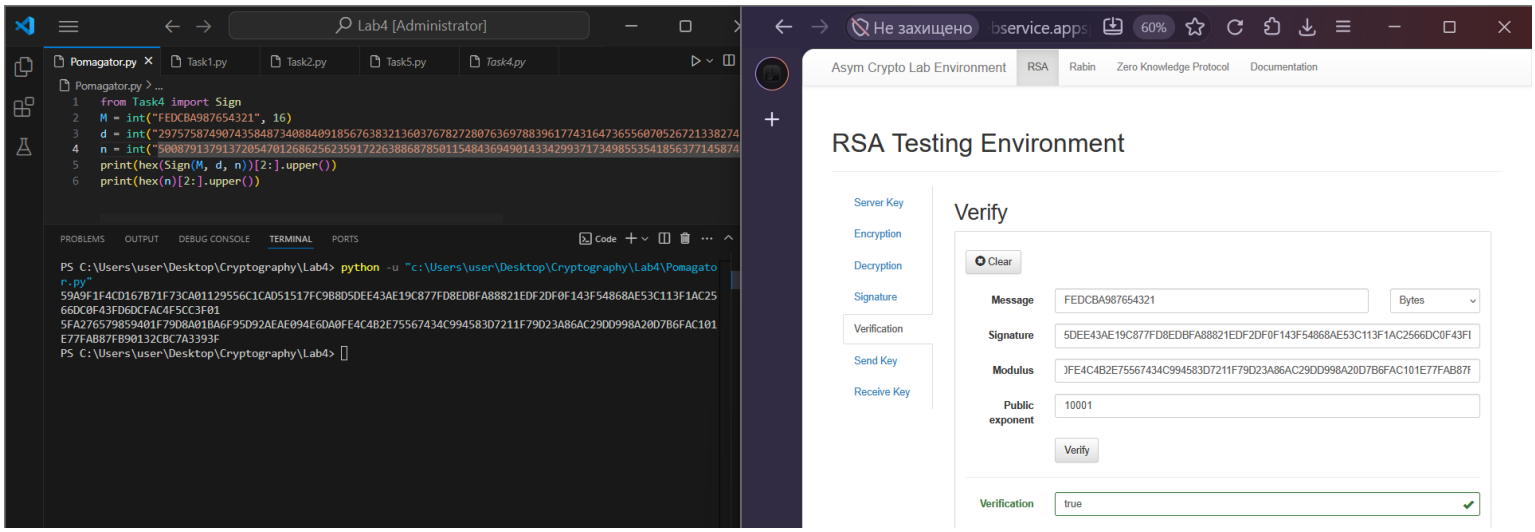
Розшифруємо у себе повідомлення відповідним d абонента А із завдання3



Перевіримо роботу цифрового підпису, підпишемо на сервері, перевіримо локально:

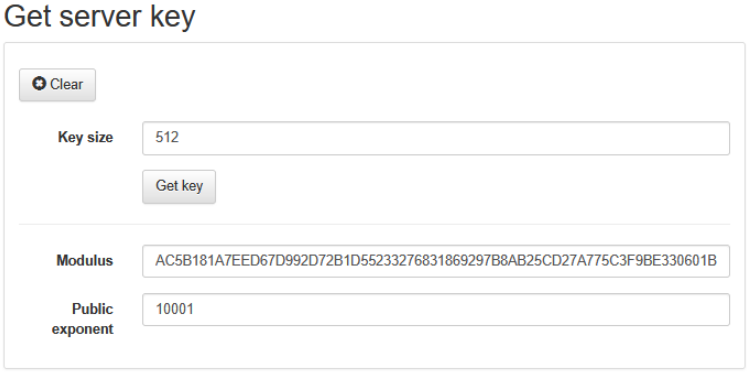


Підпишемо у себе, перевіримо на сервері

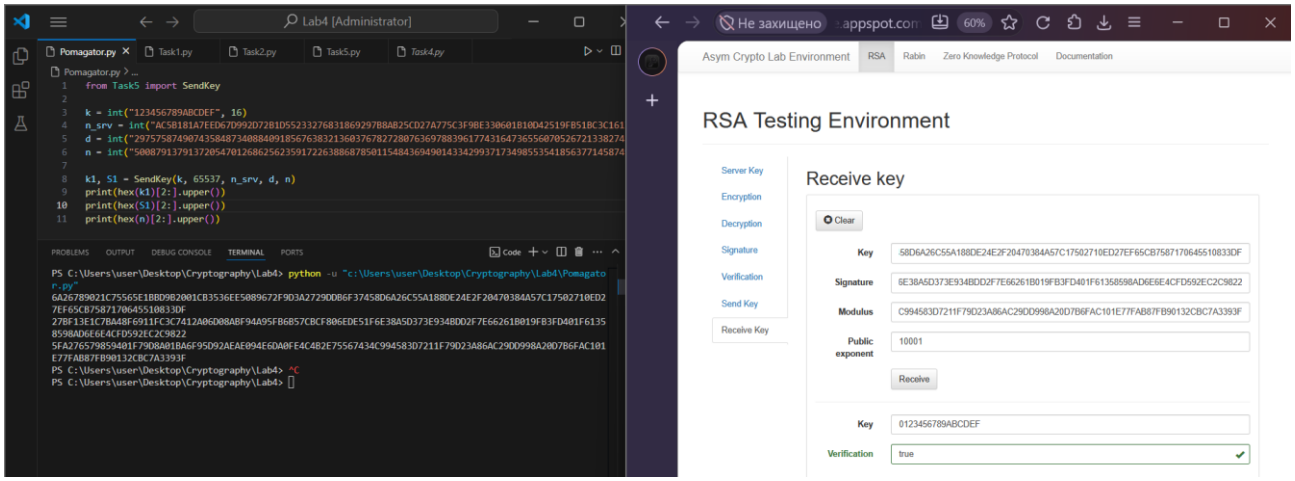


Перевіримо роботу протоколу конфіденційного розсилання ключів

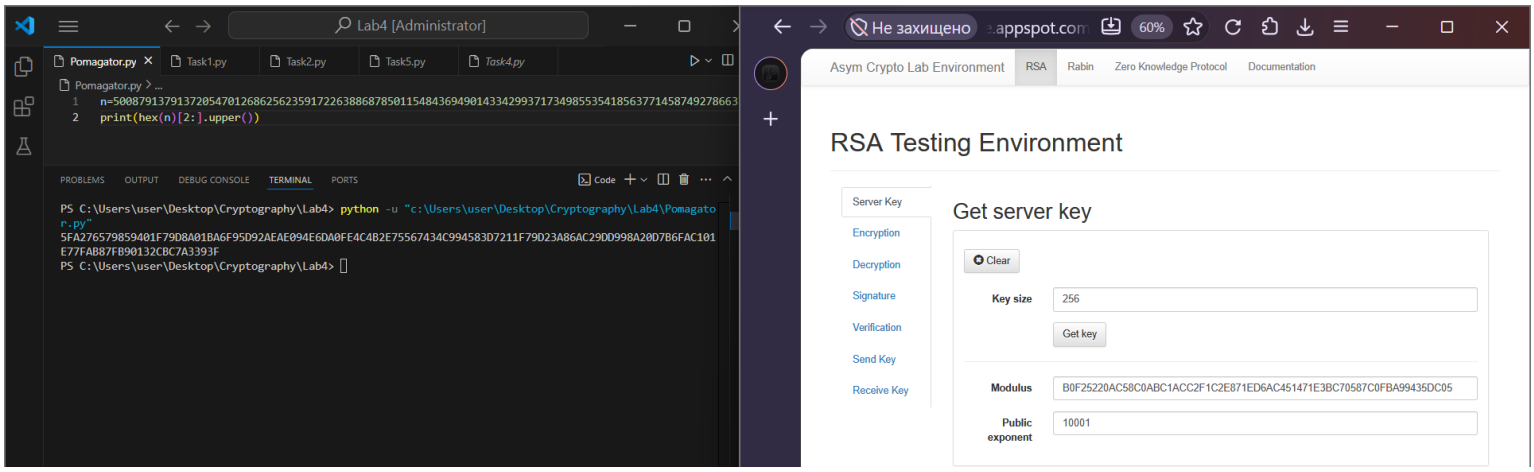
Для цього отримаємо ключ на 512 біт, аби виконувати умову n<=n_serv



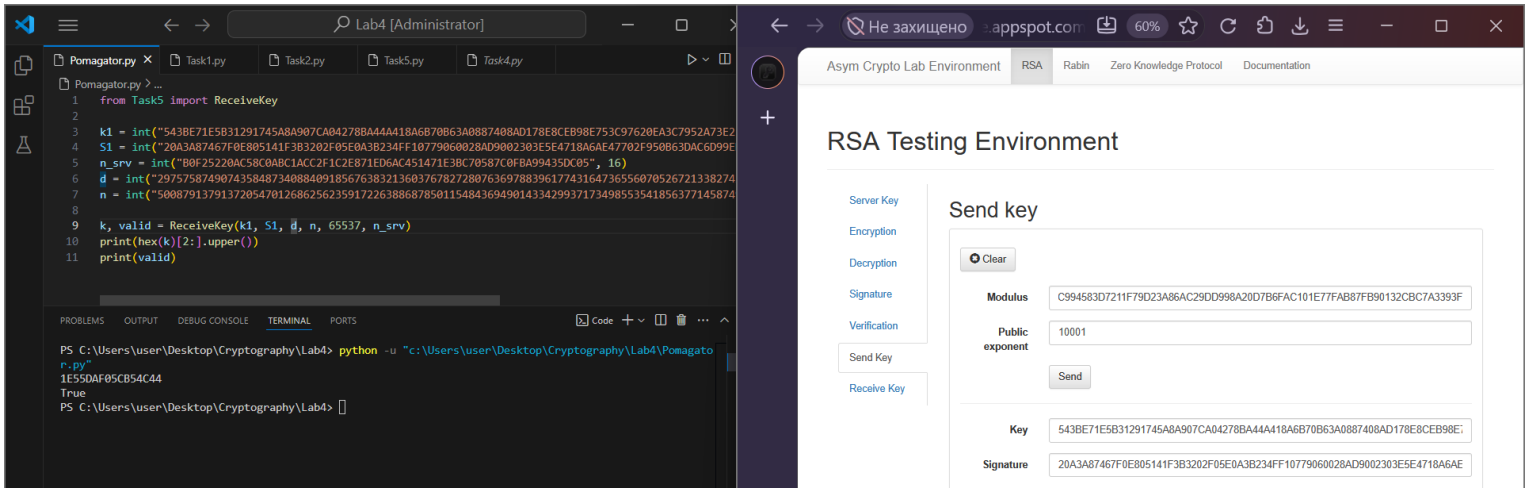
Відправимо ключ серверу, отримаємо на сайті.



Сервер відправить ключ нам. Сформуємо ключ на 256 біт, аби відповідати умові n_serv<=n, і запишемо собі знову власний hex(n)



Отримаємо ключ від сервера, ввівши записане до цього значення у n_srv



Контрольні питання

1. Які перевірки виконуються програмою під час генерації простих чисел і для чого потрібна кожна з них?

Під час генерації числа програма виконує три послідовні перевірки: встановлює старший біт у 1 для проставлення точної довжини числа 256 біт та молодший біт у 1 для забезпечення непарності, тест пробних ділень на масив ї простих чисел до 541 відсіює багато випадкових складених чисел без затратних розрахунків, тест простоти Міллера-Рабіна з 40 випадковими основами остаточно перевіряє кандидатів на відсутність нетривіальних коренів з одиниці за модулем числа, що знижує ймовірність помилкового прийняття складеного числа за просте до дуже малої величини, яка не перевищує 4^(-40).

2. Які умови повинні виконуватися для параметрів згенерованої пари ключів RSA? Як їх було перевірено у вашій реалізації?

Для параметрів RSA повинні виконуватися умови: прості p і q не повинні дорівнювати одне одному, «e» має бути взаємно простою з функцією Ейлера (gcd(e, phi(n)) == 1), «d» має бути мультиплікативно оберненою до e за модулем phi(n) (тобто (e * d) mod phi(n) == 1), а для протоколу обміну модуль відправника не повинен перевищувати модуль отримувача (n <= n1).

У нашій реалізації відмінність простих чисел виконали через while q == p, взаємну простоту через gcd(e, phi) == 1 перед обчисленням ключа d, точність знаходження оберненого через (e * d) % phi == 1, співвідношення модулів гарантували автоматичним обміном парами при n > n1, а математичну оборотність схеми перевірили контрольним шифруванням і розшифруванням числа (m^e)^d mod n == m.

3. Які дані необхідні для виконання операцій Encrypt(), Decrypt(), Sign() та Verify()? Чому кожній із цих операцій не потрібні всі параметри RSA?

Для виконання операції Encrypt(M, e, n) необхідні відкритий текст і відкритий ключ адресата (e, n). Для Decrypt(C, d, n) - шифротекст і закритий ключ адресата (d, n). Для Sign(M, d, n) - повідомлення і закритий ключ автора (d, n). Для Verify(M, S, e, n) - повідомлення, підпис і відкритий ключ автора (e, n).

Кожній операції не потрібні всі параметри через асиметричність криптосистеми та принцип мінімізації доступу: відкриті операції такі як шифрування та перевірка оперують виключно публічними даними, що дозволяє будь-кому взаємодіяти з користувачем без загрози компрометації, тоді як конфіденційні операції (розшифрування та підпис) вимагають лише закритої експоненти d, а вихідні числа p, q і значення phi(n) знищуються після генерації, бо їхнє розкриття дозволило б зловмиснику самостійно вирахувати секретний ключ d.

4. Які властивості забезпечуються на різних етапах реалізованого протоколу передачі ключа: конфіденційність та підтвердження справжності відправника?

Підтвердження справжності відправника та цілісність ключа забезпечуються на етапі створення підпису відправником шляхом обчислення S = k^d mod n на власному закритому ключі d з наступною перевіркою одержувачем рівності S^e mod n == k за відкритим ключем автора (e, n), оскільки ніхто без знання ключа d не може сформувати дійсний підпис для даного k.

Конфіденційність забезпечується на етапі шифрування даних, коли відправник шифрує сформовані значення k та S відкритим ключем адресата (k1 = k^e1 mod n1, S1 = S^e1 mod n1), а одержувач розшифровує їх своїм особистим закритим ключем d1, що унеможливлює перехоплення та читання значень сеансового ключа чи підпису стороннім спостерігачем у публічному каналі.

5. Які перевірки повинен виконати одержувач у реалізованому протоколі передачі ключа, перш ніж вважати отриманий ключ коректним і справжнім?

Одержувач після отримання зашифрованого пакета (k1, S1) спочатку розшифровує обидва значення за допомогою свого закритого ключа d1 за модулем n1, отримуючи значення k та S, після чого здійснює верифікацію цифрового підпису: підносить отримане S до відкритої «e» відправника за відкритим модулем відправника n і порівнює відновлене число з розшифрованим значенням ключа k. Якщо рівність S^e mod n == k виконується, одержувач переконується, що переданий сеансовий ключ був створений саме власником відкритого ключа (e, n) і не зазнав жодних змін чи пошкоджень під час транспортування відкритим каналом зв'язку, після чого приймає сеансовий ключ для використання.

Висновок

Під час виконання лабораторної роботи ми вивчили принципи побудови асиметричної криптосистеми RSA та реалізували генератор простих чисел на основі пробних ділень і тесту Міллера-Рабіна. За допомогою розширеного алгоритму Евкліда ми обчислили секретні ключі для двох абонентів за модулем функції Ейлера та експериментально підтвердили виконання всіх математичних співвідношень, необхідних для коректної роботи схеми.

Ми створили ізольовані процедури шифрування, розшифрування, підпису та верифікації, на основі яких побудували протокол безпечного розподілу сеансових ключів. Роботу всіх реалізованих функцій перевірили у веб-середовищі.